*Exercises · Agents and MCP*

# AI06 Exercises

Work each one before reading its solution. Exercises 2 and 5 include arithmetic. Practise Exercise 6 out loud.

## Exercise 1: Choose the rung

For each, choose a single call, a workflow (name the pattern), an agent, or multi-agent. Give a reason.

**(a)** Extract 15 fields from each invoice. **(b)** Sort incoming email into four queues, then draft a reply with a template per queue. **(c)** "Why did signups fall last Tuesday?", using logs, analytics and deploy history. **(d)** Compare 40 competitors on pricing and features, from public sources. **(e)** A loan decision that must follow a regulated sequence.

*Solution*

**(a)** Single call with a schema. There are no steps to decompose.

**(b)** Workflow: routing, then a specialized prompt per category.

**(c)** Agent. Each finding determines the next question, and errors are easy to check.

**(d)** Multi-agent, or orchestrator-workers. It fans out cleanly, and the data exceeds a single context. Confirm by eval, since it costs far more.

**(e)** Workflow with code gates. A regulated sequence must be guaranteed, so the model can help at individual steps, but mustn't choose the order.

## Exercise 2: Fix a broken loop

```
while (true) {
    Message reply = client.messages().create(params.build());
    String text = textOf(reply);
    params.addAssistantMessage(text);
    if (text.toLowerCase().contains("done") || iterations++ > 5) {
        return text;
    }
    for (ContentBlock block : reply.content()) {
        block.toolUse().ifPresent(call -> params.addUserMessage(run(call)));
    }
}
```

**(a)** List every problem. **(b)** Assume each turn adds 800 tokens to a 4,000-token start, and a task takes 12 turns. How many input tokens are billed in total, uncached?

*Solution*

**(a)**

1. It parses text for "done", so it can stop mid-task, and it can't detect finishing otherwise.
2. It treats 5 iterations as completion, and reports truncation as success.
3. It saves only text, losing `tool_use` blocks, so the next request is invalid.
4. It sends each result as a plain user message, rather than `tool_result` blocks with `tool_use_id`.
5. Separate messages discourage parallel calls.
6. It doesn't check `stop_reason`, so `max_tokens` could run truncated tool input.
7. It has no error handling for tools.

**(b)** The sum over 12 turns of $4{,}000 + 800 \times i$, for $i$ from 0 to 11, is $48{,}000 + 800 \times 66 =$ **100,800 tokens.**

## Exercise 3: Rewrite the tools

An agent has these tools:

```
get_data(id)        "Gets data."
get_info(id)        "Gets information."
do_action(id, type) "Performs an action on something."
```

It serves a telecom support desk: account lookup, usage lookup, plan changes and service cancellations. **(a)** Redesign the tools. **(b)** Write one full description. **(c)** Which action needs extra protection, and how? **(d)** How would you test it?

*Solution*

**(a)** `find_account`, `get_usage`, `change_plan`, `request_cancellation`. Each has one job, and its parameters are named: `account_id`, `from_date`, `plan_code`.

**(b)** `get_usage`: "Returns data, call and SMS usage for one account over a date range, up to 90 days. Use this for questions about a bill's usage or remaining allowance, after `find_account` has confirmed the account. Dates are YYYY-MM-DD. For plan details, use `get_plan`. Returns zeros, not an error, for a period with no usage."

**(c)** `request_cancellation`, since it's hard to undo. Add a gate requiring verified identity, ask for explicit customer confirmation, and consider human review. Enforce it in the backend as well.

**(d)** A selection eval: realistic requests with expected tools, including near-misses. Track wrong picks and invalid parameters, and keep a held-out set.

## Exercise 4: Classify the errors

For each failure, give the category, `isRetryable`, and what the agent should do.

**(a)** Payment gateway returns 503. **(b)** `start_date` is after `end_date`. **(c)** Refund window closed 12 days ago. **(d)** Service account lacks `orders:write`. **(e)** Search for an email finds no customer. **(f)** A subagent's only news source is down after 3 tries.

*Solution*

| Case | Category | Retryable | Agent should |
|---|---|---|---|
| (a) | Transient | Yes | Back off, retry, then tell the customer honestly |
| (b) | Validation | With new input | Correct the dates, or ask |
| (c) | Business | No | Explain the policy, and offer any alternative |
| (d) | Permission | No | Escalate. This is a configuration problem |
| (e) | **Not an error** | n/a | Report no match, and ask for another identifier |
| (f) | Transient, unresolved | No more local retries | Propagate the attempt, partial results and alternatives |

## Exercise 5: Budget a research system

A coordinator on Opus 5.5 ($4 / $20) uses 50,000 input and 3,000 output tokens. Each subagent on Sonnet 5.5 ($2 / $10) uses 30,000 input and 2,000 output. There are 8 subagents, in 2 rounds of 4.

**(a)** What's the cost per report? **(b)** At 500 reports a month? **(c)** What's the cost if the coordinator also runs on Sonnet 5.5? **(d)** Each subagent takes 90 seconds. What's the subagent time in sequence, and in parallel by round? **(e)** Which of these numbers can't you trust yet, and how would you decide on the model?

*Solution*

**(a)** Coordinator: $0.20 + $0.06 = $0.26. Each subagent: $0.06 + $0.02 = $0.08, so 8 cost $0.64. Total **$0.90.**

**(b)** 500 × $0.90 = **$450.**

**(c)** Coordinator: $0.10 + $0.03 = $0.13. Total **$0.77**, saving $0.13 per report.

**(d)** Sequential: 8 × 90 = **720 seconds.** Parallel, two rounds: 2 × 90 = **180 seconds.**

**(e)** Every token count is an assumption until measured. Run both coordinator models on an eval with about 20 real queries. A $0.13 saving is worthless if plans get worse.

## Exercise 6: Defend a decision

A director says: "The system prompt already says never to refund over $500. We've tested 1,000 conversations with zero violations. Hooks are over-engineering, and they'll slow us down. Remove them."

Respond in about a minute. Cover evidence, risk, a compromise, and what would change your mind.

### Model answer

"Zero in 1,000 is good, and it doesn't prove zero. Using the rule of three, zero failures in 1,000 tests only tells us that the true rate is probably below about 0.3%. At 30,000 refund conversations a month, that could still be up to 90 wrongful refunds, and test traffic isn't adversarial, as real users are.

A prompt is probabilistic. A $500 limit is a guarantee we owe to finance and auditors. The hook is a few lines of code, runs in milliseconds, and, unlike the model, doesn't vary. And the billing service should enforce the limit too, since other callers exist.

I'd keep the prompt rule, because it means the agent rarely hits the hook, and it explains the policy well. If speed of change is the concern, the limit can live in configuration.

I'd change my mind only for a rule where an occasional miss is harmless, such as tone. For money, identity or compliance, the cost of one failure outweighs the cost of a small piece of code."